

Author — Prompts Dossier

Complete prompt specifications for all 8 agents.

1. PlannerAgent

Purpose: Generate a complete, structured book outline from a user brief. This is the architectural backbone of the entire book — every other agent derives its work from the PlannerAgent's output.

System Prompt Summary:

```
You are a world-class book architect. Create a detailed, compelling book outline.  
[TONE DIRECTIVE injected based on selected tone]  
[AI-TELL REMOVAL rules]  
CRITICAL: Respond with valid JSON only matching the BookOutline schema.
```

User Prompt Template:

```
Create a complete book outline for:  
BRIEF: {user_brief}  
TONE: {tone}  
CHAPTER COUNT: {chapter_count}  
TARGET WORDS PER CHAPTER: {chapter_length}  
AUTHOR NAME: {author_name}
```

Inputs: user_brief, tone, chapter_count, chapter_length, author_name

Outputs: BookOutline JSON (title, subtitle, chapters[], glossary_seed[], callback_index[], tone{}, foreword_notes, back_cover_copy, etc.)

Failure Modes:

- LLM returns non-JSON text → stripped with `raw.find("{")` + retry
- Chapter count mismatch → padded with fallback chapters
- Missing tone block → injected from TONE_DIRECTIVES dict

Rationale: Front-loading structure prevents continuity failures downstream. Callbacks and glossary seeds are planted here so every Writer invocation has narrative targets.

2. ResearcherAgent

Purpose: Perform hybrid RAG retrieval (FAISS + BM25) and produce structured research packets with fact registries for each chapter.

System Prompt Summary:

```
You are a meticulous research agent. Extract and structure factual, high-value information.  
Output: JSON with key_facts[], citations[], fact_registry[], bm25_results[]
```

User Prompt Template:

```
Chapter Plan: {chapter_plan_json}  
Retrieved Context (RAG): {retrieved_context}  
Book Genre: {genre} | Core Thesis: {core_thesis}  
Extract structured research packets. Flag unverifiable facts as supported=false.
```

Inputs: chapter_plan_json, retrieved_context (FAISS+BM25 top-k), genre, core_thesis

Outputs: ResearchPacket JSON per chapter

Failure Modes:

- Empty retrieval → fallback_chunks used directly as key_facts
- JSON parse error → raw fallback dict returned

Rationale: Separating research from writing prevents hallucinations. The FactCheckerAgent can cross-reference against this registry. BM25 handles exact term matching; FAISS handles semantic similarity. RRF fusion gives the best of both.

3. WriterAgent

Purpose: Write full chapter prose using research facts, memory context, and callback directives.

System Prompt Summary:

```
You are a bestselling author. Your prose is your craft.  
[TONE DIRECTIVE]  
[AI-TELL REMOVAL]  
Rules: open with hook, vary sentence length, organic transitions, no bullet lists,  
end with forward-looking statement, hit target word count ±10%.
```

User Prompt Template:

```
Write Chapter {chapter_number}: "{chapter_title}"  
CHAPTER PLAN: {chapter_plan_json}  
RESEARCH FACTS: {research_facts}  
MEMORY CONTEXT: established_concepts={...}, open_callbacks={...}  
CALLBACKS TO INTRODUCE: {callbacks_to_introduce}  
CALLBACKS TO RESOLVE: {callbacks_to_resolve}  
GLOSSARY TERMS TO USE: {glossary_terms}  
TARGET WORD COUNT: {target_words}
```

Inputs: chapter plan, research facts, memory context (established concepts, open callbacks), callback directives, glossary terms, target word count

Outputs: Raw chapter prose string

Failure Modes:

- Response < 100 chars → retry; fallback_prose() used after max_retries
- Groq rate limit → exponential backoff (2^attempt seconds)

Rationale: temperature=0.8 allows creative variation. Memory context prevents continuity breaks without requiring the LLM to process the entire manuscript each time.

4. HumanizerAgent

Purpose: Rewrite AI-generated prose to remove AI-tells and achieve authentic human rhythm.

System Prompt Summary:

```
You are a master prose editor. Remove AI-tells, break formulaic patterns,
add contractions (tone-dependent), insert rhetorical questions, add domain
metaphors,
vary rhythm if 3+ sentences are the same length.
Preserve ALL factual content. Return ONLY rewritten prose.
```

AI-Tell Removal List:

- "it is important to note", "delve into", "landscape of"
- "in today's fast-paced world", "not only... but also"
- "as we explore", "in conclusion", "to summarize"
- "game-changer", "paradigm shift", "seamlessly"
- "leverage", "harness", "utilize", "ensure"

Inputs: raw_prose, chapter_number, tone, target_words

Outputs: Humanized prose string

Pre-processing: `\_count\_ai\_tells()` counts violations before/after for logging.

Failure Modes:

- Response too short → original raw_prose returned as fallback

Rationale: A dedicated humanization pass is more effective than hoping the Writer avoids AI-tells. Having an explicit list makes the violation count measurable.

5. EditorAgent

Purpose: Developmental editing — consistency, pacing, readability, grammar, tone alignment.

System Prompt Summary:

```
You are a senior developmental editor. Check pacing, tone consistency, opening hook, grammar errors, remaining AI-tells, callback integration naturalness, readability (target Flesch-Kincaid Grade 10-12 for most tones). Return polished final prose ONLY.
```

Inputs: humanized_prose, chapter_number, tone, chapter_plan_json, tone_fingerprint

Outputs: Edited prose string

Failure Modes:

- Response too short → humanized_prose returned as fallback

Rationale: temperature=0.4 (lower than Writer/Humanizer) to prevent editorial creativity overriding established voice.

6. FactCheckerAgent

Purpose: Verify factual claims against the fact registry; soften unsupported assertions.

System Prompt Summary:

```
Extract every factual claim from the prose. Cross-reference against fact registry. Supported claims: keep as-is. Unsupported claims: soften with hedging language ("research suggests", "some evidence indicates", "it's been observed that"). NEVER fabricate citations. NEVER remove claims. Preserve narrative flow. Return fact-checked prose ONLY.
```

Inputs: edited_prose, fact_registry_json (per chapter), chapter_number

Outputs: Fact-checked prose string

Failure Modes:

- Response too short → edited_prose returned as fallback

Rationale: temperature=0.2 (very low) — fact-checking is a conservative operation. Softening rather than removing preserves the author's intent while preventing misinformation.

7. MemoryKeeperAgent

Purpose: Extract structured memory from completed chapters and persist to SQLite.

System Prompt Summary:

```
Analyze completed chapter. Extract:  
- new_concepts[] with explanation and complexity  
- new_glossary_terms[] with definitions  
- callback_updates[] with resolution status  
- decisions[] with rationale  
- tone_observations{} with measurable style metrics  
Return JSON only.
```

Inputs: final_prose (first 2000 chars), chapter_number, chapter_plan_json, existing_callbacks_json, existing_glossary_json

Outputs: Memory update JSON → persisted to SQLite

Repair Logic:

- `repair\_callbacks\_for\_chapter(n)` resolves callbacks whose referenced_in_chapters includes n
- `upsert\_glossary()` uses ON CONFLICT to update definitions without duplication

Failure Modes:

- JSON parse error → empty memory update (no writes, no crash)

Rationale: Separating memory persistence from writing prevents context length explosion. The SQLite store survives process restarts, enabling chapter regeneration without losing prior state.

8. AssemblerAgent

Purpose: Generate publication-ready DOCX and PDF from all chapter content.

Front Matter Prompt Summary:

```
You are writing the {section} for a book. Match tone perfectly.  
Return ONLY the prose for the {section}.
```

Generated Sections:

- Title page
- Copyright page
- Table of Contents (manual entries)
- Foreword (LLM-generated, tone-matched)

- Preface (LLM-generated, tone-matched)
- Chapters (with chapter headings)
- Afterword (LLM-generated)
- Glossary (from SQLite store)
- About the Author (LLM-generated)
- Back Cover Copy

DOCX: Uses python-docx with custom heading styles, tab-stop TOC, section breaks, 1.25" margins

PDF: Uses reportlab with custom ParagraphStyles, page numbering via `onLaterPages` callback, styled TOC table, HRFlowable section dividers

Failure Modes:

- DOCX failure: logged, state["docx_path"] not set, pipeline continues
- PDF failure: logged, state["pdf_path"] not set, pipeline continues

Rationale: reportlab chosen over WeasyPrint to eliminate GTK system dependency issues on Windows.

Evaluation Prompt

Purpose: LLM-as-judge scoring of the generated book.

System Prompt Summary:

```
You are an expert book evaluation judge. Score on 6 metrics (0-10 each).  
Be honest, rigorous, specific. Output JSON with metrics[], overall_score,  
llm_judge_verdict, recommendations[].
```

Metrics:

1. structural_completeness
2. tone_fidelity
3. ai_tell_detection (higher score = fewer tells)
4. callback_consistency
5. fact_grounding
6. readability

Inputs: title, tone, chapter_excerpts (first 2000 chars × first 3 chapters), callback_index

Failure Modes: Empty metrics → 6 placeholder metrics at 5.0 inserted

Rationale: Providing only excerpts (not full book) avoids context length limits while still giving the judge enough material. Rubric scoring with evidence quotes makes results actionable.